AI8HT

ONE PLATFORM · TEAM HANDBOOK

Setup & Work Assignment

Fork it, run it, and know exactly what you are building

Everything each of the five of us needs to start: how to get the application running on your own machine, what you personally own and build, how to commit so we can always tell who did what, and where your work goes. Nothing anyone builds reaches the public site until it has been reviewed and released deliberately.

Version **1.2 · 5 September 2026**	Repository **github.com/PVUD14/ONE-V4**	Team **Divakar, Chandra, Manisha, Navya, Santhosh**
You work from **Your own fork**	Production **one.ai8ht.pro — Divakar only**	AI / LLM work **On hold until the GPU exists**

SECTION 01

Five things that are true before you write any code

1 · ONE REPOSITORY, AND YOU FORK IT

`PVUD14/ONE-V4` is the only repository from now on. You have **read** access to it and you work from your own fork — you cannot push to it, by design. The older `planthepath` and `planthepath-next` repositories are history; do not clone, push to, or take code from them.

2 · TWO BRANCHES THAT MATTER

In the upstream repository, `main` is exactly what is running on one.ai8ht.pro right now, and `integration` is where the team's work comes together. You branch from `integration` in your fork and open pull requests back into upstream `integration`.

3 · NOTHING YOU PUSH GOES PUBLIC

Your pushes land in your own fork. Once a pull request merges into upstream `integration` it deploys to the AWS staging environment, where we all review it. Only Divakar promotes to `main`, and only Divakar releases to the public site, by hand.

4 · THE LOOK IS FROZEN

Stylesheets, design tokens and the AI8HT wordmark are not to be modified. Build with the components and classes that already exist. If something needs a style change, ask Divakar.

5 · AI AND LLM WORK IS ON HOLD

Nobody integrates a model until the GPU instance exists. We are building the *shape* a model will slot into: stored rows, jobs on a queue, and a review step. Details are in your own assignment section and in the Parallel Build Plan.

Where your work goes

Your fork — you push here → **upstream integration** — pull request, peer reviewed → **AWS staging** — automatic, we all check it → **one.ai8ht.pro** — Divakar approves & releases

Why a fork, and not a branch in the main repository?

Because a rule you cannot break is worth more than a rule you agree to follow. GitHub does not enforce branch protection on a private repository under a personal account, so "nobody pushes to `main`" would have been a promise rather than a control. With read access and your own fork it becomes a permission: you *cannot* write to the upstream repository, and neither can any agent working on your behalf, whatever either of you is asked to do. Everything arrives as a pull request, which is what we wanted anyway.

Never do any of these. They apply to you and to any AI agent working on your behalf.

Deploy or connect to the production server · copy production data · commit a key, token or password · force-push a shared branch · edit or delete an existing migration · modify a frozen style file or the brand assets · remove or reorder navigation items · edit files another person owns · disable a test or an authorization check · put invented data on a screen a real user can reach. The full list, and why each matters, is in the *Contributor & Agent Rules* document. Read it once before your first commit.

If you use Codex, Claude Code or any AI coding assistant.

That is expected and fine. Two conditions. First, the repository carries an `AGENTS.md` and a `CLAUDE.md` at its root — your agent reads these automatically and they contain the rules; do not delete or edit them. Second, you remain accountable for everything your agent writes: read the diff before you commit it, and record the tool in your commit trailer (Section 03). An agent that produces a change you have not read is not a shortcut, it is an unreviewed change with your name on it.

SECTION 02

Set up your machine

Do this once, in order. It takes about twenty minutes on a clean machine. Everything below has been checked against the actual repository — if a step fails, say so rather than improvising, because a workaround that only exists on your laptop is a problem for everyone later.

1 Install the toolchain

Node.js 20 — production runs 20.20.2, so match the major version. **Git. Docker** (for a local PostgreSQL, unless you already run Postgres 16 natively). Then enable the package manager the repository pins:

```
corepack enable
corepack prepare pnpm@10.15.0 --activate
node -v      # expect v20.x
corepack pnpm -v   # expect 10.15.0
```

2 Fork the repository

Open `https://github.com/PVUD14/ONE-V4` and click **Fork**. You have read access to the upstream repository and cannot push to it — your fork is where your work lives. Ask Divakar if you have not been added as a collaborator, or if the Fork button is not offered.

3 Clone your fork and connect it to upstream

Two remotes: `origin` is your fork and is the only place you push; `upstream` is the team repository you pull from.

```
git clone https://github.com/<your-username>/ONE-V4.git
cd ONE-V4
git remote add upstream https://github.com/PVUD14/ONE-V4.git

git fetch upstream
git checkout -b integration upstream/integration

git remote -v      # origin = your fork, upstream = PVUD14/ONE-V4
```

Never work directly on `main` — it represents the live site.

4 Install dependencies

```
corepack pnpm install
```

This is a pnpm workspace: one install at the root covers the API, the web app and all nine packages.

5 Start a local PostgreSQL 16

Production runs PostgreSQL 16, so use 16 locally. The connection details below match what the example environment file expects, including the non-standard port.

```
docker run -d --name one-db -p 55432:5432 \
  -e POSTGRES_USER=planthepath \
  -e POSTGRES_PASSWORD=local \
  -e POSTGRES_DB=planthepath_dev \
  postgres:16-alpine
```

6 Create your environment file

Copy the example and set the database password you just used. **Your `.env` is yours alone** — it is git-ignored, and it must never be committed or shared.

```
cp .env.example .env
# then edit .env and set:
DATABASE_URL=postgresql://planthepath:local@127.0.0.1:55432/planthepath_dev
```

Leave the Cognito values as the examples for now. Sign-in against real identities is not needed for most local work, and production identity resources are never used locally.

7 Load the environment into your shell

The API reads plain environment variables — it does not load `.env` by itself. Run this in each terminal before starting the API or the web app.

```
# Git Bash, macOS, Linux
set -a; . ./.env; set +a

# Windows PowerShell
Get-Content .env | ForEach-Object {
  if ($_ -match '^\s*([A-Za-z_][A-Za-z0-9_]*)=(.*)$') {
    Set-Item -Path "env:$($Matches[1])" -Value $Matches[2]
  }
}
```

8 Create the schema and add starter data

Sixteen migrations build the 109-table schema. The seeds give you content to look at — **seeded data only; never a copy of production.**

```
corepack pnpm db:migrate
corepack pnpm --filter @planthepath/persistence db:seed:local-users
corepack pnpm --filter @planthepath/persistence db:seed:careers
```

The career-readiness content seed publishes governance-attributed content, so it needs an existing author, reviewer and publisher — three *different* accounts, because the database rejects an approval by the author of the revision. `db:seed:local-users` creates them and prints the three ids ready to paste:

```
# Git Bash, macOS, Linux
export CAREER_READINESS_AUTHOR_USER_ID=11111111-1111-4111-8111-111111111111
export CAREER_READINESS_REVIEWER_USER_ID=22222222-2222-4222-8222-222222222222
export CAREER_READINESS_PUBLISHER_USER_ID=33333333-3333-4333-8333-333333333333

# Windows PowerShell
$env:CAREER_READINESS_AUTHOR_USER_ID    = '11111111-1111-4111-8111-111111111111'
$env:CAREER_READINESS_REVIEWER_USER_ID  = '22222222-2222-4222-8222-222222222222'
$env:CAREER_READINESS_PUBLISHER_USER_ID = '33333333-3333-4333-8333-333333333333'
```

```
corepack pnpm --filter @planthepath/persistence db:seed:career-readiness
```

Do not invent your own UUIDs here. The seed checks the format but not existence, so a made-up id gets past the check and then fails on a foreign key against the users table — a confusing way to lose ten minutes.

9 Run it — two terminals

```
corepack pnpm api:dev    # terminal 1 -> http://127.0.0.1:3000
corepack pnpm web:dev    # terminal 2 -> http://127.0.0.1:5173
```

Remember to load the environment (step 7) in *both* terminals.

10 Confirm it works

```
curl http://127.0.0.1:3000/health     # expect {"status":"ok"}
corepack pnpm typecheck
corepack pnpm test
```

Then open `http://127.0.0.1:5173`. You should see the login page with the AI8HT wordmark. With `VITE_ENABLE_DEMO_PORTAL=true` in your `.env`, `/demo` gives you the full role previews to explore locally — that flag is for local work only and is off in production.

Typecheck and tests must pass before you start changing things. If they fail on a clean clone, that is a problem for the whole team — report it, do not work around it.

> **Stuck on setup?**
> Ask in the team channel rather than spending a second day on it. A setup problem one person hits is usually a setup problem everyone will hit, and it is worth fixing in this document rather than five times in private.

SECTION 03

Your daily loop — and where to upload

The same six steps every time. The only place your work goes is a pull request from your fork into `integration` on ONE-V4.

1 Start from a fresh upstream integration, on your own branch

Always start from `upstream`, never from whatever your fork happened to have. Your fork does not update itself.

```
git fetch upstream
git checkout integration
git reset --hard upstream/integration
git checkout -b stream/<letter>-<yourname>/<short-topic>

# examples
git checkout -b stream/b-manisha/employer-interviews-view
git checkout -b stream/c-navya/staff-action-record
git checkout -b stream/d-santhosh/readiness-band-ring
```

Your stream letter is on your assignment page. One branch per piece of work, not one branch for the month.

2 Work only in the files your stream owns

Your assignment page lists them. If your task needs a change in someone else's file, message that person — do not edit it, and do not let an agent edit it for you. Never reformat a file you do not own; a tidy-up pass turns everyone else's clean merge into a manual one.

3 Commit with the trailer — every time

This is how we answer "who did what" months from now without keeping a spreadsheet. A blank line, then the block, at the end of the message.

The block must be the last paragraph, with no blank line inside it. Git parses only the final paragraph as trailers, so a blank line before any of these lines — including before a `Co-Authored-By` — silently turns all of them into ordinary body text and the attribution report returns nothing. Check with `git log -1 --format='%(trailers:key=Stream,valueonly)'`; if it prints nothing, your block is broken.

```
feat(institution): add staff action record and intervention writes

One staff_actions table with a type discriminator, serving interventions,
tasks, notes, verification and meetings.

Stream: C — Institution
Owner: Navya
Agent: Codex (gpt-5-codex)            # or: none
Agent-Scope: drafted implementation   # omit if Agent: none
Reviewed-By: pending
```

Be honest in the `Agent` field. It is not about credit or blame. If an assistant turns out to have repeated a wrong assumption across twenty files, that one field turns a week of searching into a single query.

4 Put a header on every new file you create

```
/**
 * ONE · Stream C — Institution
 * Owner: Navya · Created: 2026-09-12
 * Agent-assisted: yes (Codex) · Reviewed: pending
 *
 * One staff-action record serving interventions, tasks, notes,
 * verification and meetings across four institution roles.
 */
```

Use `--` comments for SQL migrations. Once written, the header stays — do not strip it in a later edit.

5 Check before you push

```
corepack pnpm typecheck
corepack pnpm test
git diff upstream/integration...HEAD --stat   # read this. is it only your files?
```

6 Push your branch and open the pull request

```
git push -u origin stream/<letter>-<yourname>/<short-topic>
```

`origin` is your fork, so this pushes to your own copy. Then open a pull request on GitHub **from your fork's branch into `PVUD14/ONE-V4` → `integration`** — never into `main`. GitHub offers this automatically after the push, or use the "Contribute" button on your fork. Title it `[C · Navya] Staff action record and intervention writes`. The body must state:

- Stream, Owner, Agent — the same values as your commit trailer
- What changed and why, in two or three sentences
- Screens affected, or "none"
- Migrations added — the numbers, from your own block
- Style files touched — must be "none" unless Divakar approved it
- What you actually tested

Another stream owner reviews it. Once merged, it appears on AWS staging within minutes and the whole team can look at it.

> **That is the end of your part.**
> You never deploy, and you never touch the public site. Promotion from `integration` to `main` is a separate pull request that only Divakar approves, and even merging to `main` does not deploy anything — the production release is a manual step Divakar performs from a runbook, with a rollback target recorded first.

Merge small, merge often

A branch that has not merged in a week is about to cause somebody a bad day. Rebase on upstream daily — `git fetch upstream && git rebase upstream/integration` — so that conflicts arrive one at a time while they are still small. Note this is `upstream`, not `origin`: your fork is only as current as your last fetch.

Migration numbers — take only your own

Migrations are numbered in sequence, and two people creating `0017` in the same week is a guaranteed conflict. Blocks are reserved. Never take a number outside your block, and never edit or delete a migration that already exists — they have already run in production.

OWNER	YOUR BLOCK	USE IT FOR
Divakar	0016, 0028+	Membership role trigger; anything cross-cutting
Manisha	0017–0019	Interview and offer read models, talent pool
Navya	0020–0023	Staff-action record, staff lifecycle
Santhosh	0024–0027	Content tables, generation jobs, experience records
Chandra	—	Infrastructure stream; no application migrations

SECTION 04

Chandra — Stream A · Platform

Your stream touches no application source, which means you are never waiting on anyone's review and nobody is waiting on yours. You also own the environment the rest of the team's work lands in — so staging existing early unblocks everybody.

Chandra

Stream A · Platform, AWS and GPU readiness

BRANCH PREFIX	MIGRATION BLOCK	UPLOAD TO
`stream/a-chandra/…`	None — no app migrations	PR into `integration`

Files you own

`deploy/` · `.deploy/` · `infra/` · `.github/workflows/` · server configuration. No application source.

What to build, in this order

A1 **Rotate the backup storage key — today, before anything else**
The object-storage key that grants write and delete on our off-site backups has been disclosed. Everything else waits behind this.

A2 **Stand up AWS staging on the t3.medium instance**
PostgreSQL 16 from the PGDG repository to match production exactly — not the distribution default, which is 18. Plus nginx, certbot, and ClamAV with its signature updater *enabled at boot* (on the old host it runs but is disabled, so signatures go stale after a reboot). Add swap; there is none today and the scanner holds close to a gigabyte.

A3 **Isolate staging properly**
Its own database, its own secrets, access protection so it is not publicly browsable, no search indexing, and a permanent STAGING banner driven by an environment flag so nobody can mistake it for the live site. Seeded data only — production data never leaves production.

A4 **Auto-deploy `integration` to staging**
The moment a pull request merges, the team should be able to look at it. This is the single change that makes the review gate work.

A5 **Move builds off the application host**
Two vCPUs should not run a production build while serving traffic. Extend the existing CI workflows to produce the release artifact.

A6 **Rotate the remaining disclosed secrets**
Database password, both monitoring ping URLs, and the automation signing secret. The file-encryption key is carried verbatim — it cannot be rotated without re-encrypting stored documents, and that is a separate job.

A7 **Monitoring and verified backups on staging**
The four systemd units and their timers. A dump that has never been restored is not a backup — keep the restore-verification behaviour.

A8 **Add the four CI guards**
Extend `foundation-policy.yml`: fail a pull request that touches a frozen style file without approval; fail a commit missing its Stream/Owner/Agent trailer; fail a migration number outside the author's block or an edit to an existing migration; extend secret scanning beyond private keys to tokens and connection strings.

A9 **Write the GPU provisioning runbook**

with Divakar

Instance family and drivers, an inference runtime exposing an OpenAI-compatible endpoint, a private network path so the model is never publicly reachable, model storage sizing, and a written cost ceiling with an idle-shutdown rule.

Done when

Staging runs the team's work automatically and safely; disclosed secrets are rotated; the CI guards enforce the rules without anyone policing by eye; and a one-page GPU runbook exists that can be executed the day the instance is approved.

One constraint to design around.

Staging and production must never share a machine. The AWS box has 2 vCPU and under 4 GB, most of a gigabyte of which the virus scanner holds resident — it can host one or the other. When production eventually moves to AWS, it moves to its own instance and staging keeps this one.

SECTION 05

Manisha — Stream B · Hiring

Hiring is the most complete backend we have and the most under-exposed. Eleven endpoints are implemented, enforced and audited; the interface reaches them through two screens. Most of your stream is turning work we have already paid for into visible product.

Manisha

Stream B · Employer and student hiring

BRANCH PREFIX	MIGRATION BLOCK	UPLOAD TO
`stream/b-manisha/…`	0017 – 0019	PR into `integration`

Files you own

Employer and student hiring controllers · the hiring package · the hiring pages · the employer branch of the portal dispatcher.

What to build, in this order

B1 **Re-point the Interviews and Offers sidebar items — no backend needed**
Both actions already work from the applicant row, yet both sidebar items tell recruiters the feature is "not built". Point them at the Applicants screen with a pre-applied state filter, exactly as Talent Pool already does. Hours of work; ends a false statement the product currently makes to every recruiter.

B2 **Add the two missing list endpoints**
Interview create, state change and outcome all exist — nothing lists interviews. Offer extend and withdraw exist — nothing lists offers. The applicant record carries only a count and a state string. Migration 0017 if you need read models.

B3 **Build the Employer Interviews view**
List, reschedule, change state, record an outcome. Two of those three mutations have no interface anywhere today despite being fully implemented.

B4 **Build the Employer Offers view**
List, track the candidate response, withdraw.

B5 **Replace two placeholders hiding inside a live workflow**
The scheduler hardcodes a one-hour video slot at 10:00 one week out, and the offer action collects compensation through a browser prompt dialog. Both run in real, audited production code. Production currently holds zero interviews and zero offers, so there is nothing to regress — this is the moment.

B6 **Student Interviews**
Employers schedule real interviews against a student's application and the student has no way to see them. Add interview fields to the student application read model, then the student-facing view.

B7 **Talent Pool as a real saved pool**
Today it is an alias of the applicant list for one opportunity. Migration 0018 or 0019.

Done when

A recruiter can run a hire end to end from dedicated screens — post, publish, shortlist, schedule, record an outcome, offer, withdraw — and the student sees their interviews and responds to offers.

> **Watch the consent behaviour.**
> The applicant list already stops sharing a withdrawn candidate's details with the employer. That is required behaviour, not incidental — preserve it in every new view you build.

SECTION 06

Navya — Stream C · Institution

Faculty and Department Head have 29 screens between them and 21 are read-only. That is not 21 missing features — it is one missing concept wearing five costumes. Your stream builds that one concept and lights up four roles at once.

Navya

Stream C · Institution staff

BRANCH PREFIX	MIGRATION BLOCK	UPLOAD TO
`stream/c-navya/…`	0020 – 0023	PR into `integration`

Files you own

A new staff-actions controller · the institution onboarding controller · the institution branch of the portal dispatcher · staff pages and config · the Interventions page.

What to build, in this order

C1 **Wire Institution Reports — no new backend needed**

An institution career-readiness report endpoint already exists and is callable. The Reports screen is a read-only placeholder that simply never calls it. Pure wiring, and a good first merge.

C2 **Design the staff-action record and circulate it before writing the migration**

One table with a type discriminator, not five feature-specific tables: *actor, subject student, type, state, note, due date, audit trail.* Three roles depend on getting this shape right, so get eyes on it first.

C3 **Migration 0020 — the staff_actions table**

Model the review/state pattern on the assessment question bank, which already does this well. Do not invent a second pattern.

C4 **Make Interventions real for all four institution roles**

The screen is already a pilot-exact shell, and part of it is already live — the queue of students who have not attempted an assigned assessment comes from real results. Keep that, and fill the rest from the new table.

C5 **Assigned Tasks, Faculty Notes, Academic Verification and Meetings**

All four on the same table, distinguished by type. This is where 21 read-only screens start becoming usable.

C6 **Complete the staff lifecycle**

Invitation works end to end today. Revoke, resend and end-membership each need an endpoint and a control — the Staff screen already names its own gap in plain text.

C7 **Placement Phase 1 — promote five screens from demo to real**

Students, Departments, Batches, Student Search and Readiness already exist as a demo design and map onto institution endpoints the Administrator already uses. The Placement Officer's usable portal goes from three screens to eight with no new tables.

Done when

Faculty and Department Head can each record something a colleague can see and act on, and the Placement Officer portal is no longer three screens.

> **Held — do not start Placement Phase 2.**
> Pipeline, Applications, Interviews, Offers and Employers all require the institution to read employer-side application data, and no controller offers such a path. That is a consent and disclosure decision Divakar must write down first: *what may a college see about its own student's application to a company, and on whose consent?* Building it before the rule exists produces a data path we would have to withdraw.

SECTION 07

Santhosh — Stream D · Learner

Yours is the stream that makes the GPU worth having. Five student screens are finished shells whose eventual content source is a model. You are building their tables, contracts and review pipeline now, filled by seeding — so that GPU day switches five features on rather than starting them.

Santhosh

Stream D · Learner experience and the generation pipeline

BRANCH PREFIX	MIGRATION BLOCK	UPLOAD TO
`stream/d-santhosh/…`	0024 – 0027	PR into `integration`

Files you own

The five student shell pages · the student dashboard · the career-readiness panel · student pages · a new content controller and content package.

What to build, in this order

D1 **The readiness band ring — presentation only, the engine already runs**

Do not build a scoring engine. It exists and has already produced snapshots in production: an active model version, seven readiness snapshots, fourteen factors, fourteen mastery projections over seventy-one evidence records — and the learner endpoint already returns the band. Replace the NA/100 ring with a four-segment band ring, the current band highlighted, the existing readiness factors as the legend, and the four category rings driven by the per-topic mastery projections. It lands on the student dashboard and all four staff dashboards at once.

D2 **Migration 0024 — content tables with the generation columns**

Every content row carries `source`, `generator_version`, `prompt_version`, `review_state` and `generated_at`. Today only `SEEDED` and `AUTHORED` occur; the screen renders all sources identically.

D3 **Generation events on the existing outbox rail, with a non-AI worker**

Add `content.generation.requested` and `content.generation.completed`, and a worker that fills rows from seed data. This proves the whole pipeline end to end before a model exists.

Never a call from a controller

— a request handler must never wait on generation.

D4 **The review queue**

Rows land as `DRAFT`; a person publishes them. Model it on the assessment question bank, which already implements exactly this. Have reviewers publish seeded content through it so the queue is proven before a model ever writes to it.

D5 **Fill the five shells with real, non-AI content**

Practice, Company Prep, Career Tools, Scholarships, Resume Builder. Seeded and imported content, modest volume — the point is that the pipeline is exercised, not that the library is large. Practice already shows a real streak; keep it.

D6 **Student Experience records**

(0025)

Projects, internships, portfolio. The one student screen with no shell at all today.

D7 **The AI Career Coach page**

Built over the assistant that already works as a floating widget on every student screen. A page, not a new assistant.

Done when

No readiness ring anywhere says NA, and all five shells serve real content through a pipeline whose only missing participant is the generator.

> **Held — no model work.**
> No prompt engineering, no model calls, no generated content, before the GPU exists. The test for every decision you make: *if a GPU appeared tomorrow, what would I have to change?* If the answer includes a schema change, a screen change or a controller change, the design is wrong. The only acceptable answers are: deploy a worker, set a configuration value, turn on a flag.

SECTION 08

Divakar — Stream E · Core

Three streams are shaping work around decisions only you can take, so week one is those decisions and nothing else. Beyond that you own the model provider seam — the piece that decides whether GPU day costs a day or a quarter — and the release path.

Divakar

Stream E · Core, AI seam, decisions and release

BRANCH PREFIX	MIGRATION BLOCK	ALSO
`stream/e-divakar/…`	0016, then 0028+	Sole approver for `main` and production

Files you own

The assistant module · the application shell · API client structure · the authorization package · the membership role trigger · **all frozen style and brand files** · the migration registry.

Week one — because others are waiting

E1 **Write down the four role decisions**

Recommended: enable Interviewer scoped to three screens; fold Hiring Manager into Recruiter; fold Career Counselor into institution staff; drop Institution Analyst as a role and make analytics-and-export a permission. This turns 45 unreachable screens into roughly three to build.

E2 **Write the institution disclosure rule**

What a college may see about its own student's application to an employer, and on whose consent. Navya's Placement Phase 2 cannot start without it.

E3 **Sign off the readiness presentation**

Band ring, or a numeric projection that would fight a deliberate database constraint. Santhosh starts on your answer.

E4 **Commit the guard-rail files**

`AGENTS.md` and `CLAUDE.md` so every agent session loads the rules automatically; `CODEOWNERS`; a pull request template. Add `*.pem` and `*.key` to `.gitignore`.

E5 **Set up the fork-based access model**

GitHub does not enforce branch protection on a private repository under a personal account, so protection comes from permissions instead: the team gets

read

access to ONE-V4 and works from their own forks. Confirm forking is enabled, disable squash merging so commit trailers survive, and keep the ruleset in place so it activates automatically if the repository ever moves to an organisation.

E6 **The Phase 0 file split**

Three files are touched by every stream: the portal dispatcher (837 lines) into three modules by portal family; the API client (974 lines) into per-domain modules; and the Interventions page out of the shared pilot-pages file. One small pull request everyone rebases onto. A day now, versus a weekly merge conflict for three months.

The AI seam — built during the hold

E7 **Add a base URL and provider adapter to the assistant service**

It already injects its key, model and HTTP implementation and fails closed when unconfigured. These two additions make a self-hosted OpenAI-compatible endpoint a configuration value rather than a rewrite. Note the assistant is currently dark in production — its key is not set on the server.

E8 **Move the rate limiter out of process memory**

The daily per-user cap lives in a map inside one process; it survives neither a restart nor a second instance.

E9 **Own prompt governance and the grounding corpus**

Versioned prompts, the hard boundaries already written into the system prompt, and the rule that every generated row is traceable to the prompt that produced it.

E10 **The GPU cutover runbook**

with Chandra

So switching on is executed, not improvised. Start with the assistant — it is finished, grounded and capped, and it is the cheapest real test of the GPU and the network path.

Ongoing

Migration 0016 for the role trigger once the decisions land. Review every pull request. Approve promotions from `integration` to `main`. Perform every production release by hand from the runbook, recording the rollback target first. Keep the reality matrix current — it is already a release behind, and that is how it got there.

SECTION 09

Quick reference

The page to keep open while you work.

Everything at a glance

PERSON	STREAM	BRANCH PREFIX	MIGRATIONS	FIRST TASK
Chandra	A · Platform	`stream/a-…`	—	Rotate the backup storage key
Manisha	B · Hiring	`stream/b-…`	0017–0019	Re-point Interviews & Offers sidebar
Navya	C · Institution	`stream/c-…`	0020–0023	Wire Institution Reports
Santhosh	D · Learner	`stream/d-…`	0024–0027	Readiness band ring
Divakar	E · Core	`stream/e-…`	0016, 0028+	Write the four role decisions

The commands you will use every day

```
git fetch upstream
git checkout integration && git reset --hard upstream/integration
git checkout -b stream/<letter>-<yourname>/<topic>

# ... work ...

corepack pnpm typecheck && corepack pnpm test
git diff upstream/integration...HEAD --stat   # only your files?
git commit                                    # with the trailer block
git push -u origin stream/<letter>-<yourname>/<topic>
# then open a PR from your fork into PVUD14/ONE-V4 -> integration
```

Before every commit

CONTENT

- Only files my stream owns are changed.
- No frozen style or brand file touched.
- No navigation item added, removed or reordered.
- No invented data — NA where a source is missing.
- No secret, key or token in the diff.
- No test, check or guard weakened.
- Migration number inside my block; no existing migration edited.

FORM

- Branch is `stream/<letter>-<name>/<topic>`.
- Trailer carries Stream, Owner, Agent, Reviewed-By.
- `Agent` is accurate — the real tool, or `none`.
- Every new file has its header banner.
- PR title is `[Stream · Owner] …`, from your fork into upstream `integration`.
- Typecheck and tests pass locally.
- Branch rebased on `upstream/integration`.

Who to ask

QUESTION	ASK
Can this go to the public site?	Divakar — the only approver, and the only person who releases
I need a style or brand change	Divakar. Do not work around it locally.

QUESTION	ASK
I need a file another stream owns	That stream's owner. Open an issue; do not edit it yourself.
I need a migration number outside my block	**Divakar**, who holds the registry
Staging is broken, or my merge did not deploy	**Chandra**
Setup will not work on my machine	The team channel — it is probably everyone's problem
A rule is blocking legitimate work	**Divakar** — a scoped exception, or the rule changes and is reissued

> **Three documents, and what each is for.**
> This handbook tells you how to set up and what to build. The *Contributor & Agent Rules* is the binding rulebook — read it once before your first commit, and feed it to your agent if the repository rule files are not yet in place. The *Parallel Build Plan* explains why the work is split this way and how it reaches the GPU. If any of them disagrees with the repository, the repository is right and the document is out of date — say so and it will be reissued.